Grid Queue Forecaster: Project Plan

Oct 20, 2026 · @kevin

Problem and why now

Most projects in US interconnection queues never get built, so the headline queue total overstates future power supply. AI data-centre demand makes that gap matter: utilities, grid operators and AI companies need a realistic supply number, not the queue total.

The question: of the projects queued today, how much will realistically get built, where, and when?

What's new here: Berkeley Lab already publishes aggregate completion rates. This project adds project-level odds, a backtest proving the odds are calibrated, and regional capacity ranges.

Goal, audience, non-goals

Goal: by Oct 24, 2026, ship a public repo that forecasts how much queued US capacity gets built, by region and year, with calibrated uncertainty.

Audience

- **Primary:** recruiter and interviewer for the OpenAI io Software TPM intern role. They should get the finding in 2 minutes from the README.
- **Secondary:** an energy analyst. The method has to survive their scrutiny.

Non-goals for v1

- Data-centre demand side (v2)
- ML model comparison (stretch goal)
- Web dashboard or app
- Policy recommendations
- Forecasting projects that haven't entered a queue yet

Success criteria

The project is done when every row below passes. Targets marked proposed can be adjusted on Day 1.

Area	Target	How it's checked
Data	Project counts by status and region match the Berkeley Lab report within 2% (proposed)	Compare against the report's tables
Model	Build and withdrawal curves for every technology and region group	Plots generated from code
Backtest	Predicted vs actual build rates within 5 percentage points per bin (proposed); beats a naive average baseline	Calibration plot and Brier score
Outlook	P10, P50 and P90 new MW by region and year, 2026 to 2030	Monte Carlo output table and chart
Quality	Tests on cleaning and model logic all pass	pytest
Shipping	Public repo with README and one headline finding	Repo link works logged out
Ownership	Kevin explains every decision in 2 to 3 sentences	Explain-back at end of each day

Scope

In (v1)

- Every US queue in the Berkeley Lab file, grouped by grid region
- Technologies: solar, storage, wind, gas, hybrid; everything else grouped as other
- Three outcomes: built, withdrawn, still waiting
- Capacity outlook for 2026 to 2030

Out (v2 or later)

- Data-centre demand and the supply gap map
- ML model comparison (random survival forest vs classical baseline)

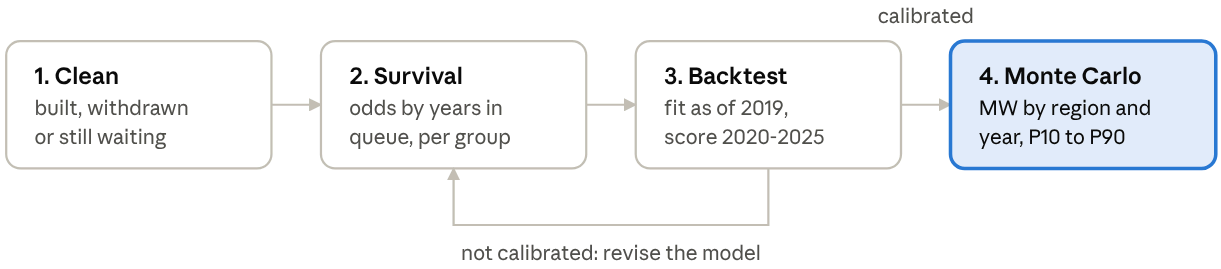
- Transmission upgrade costs and project features not in the Berkeley Lab file
- State-level outlook, if Day 2 runs late

Approach

Four stages, each with one job. The backtest gates the outlook: if the model isn't calibrated, we fix it before forecasting.

1. **Clean and define outcomes.** Map every project to built, withdrawn or still waiting, with a date.
   - Why: queue rules differ by region. One consistent definition is what makes regions comparable.
2. **Competing-risks survival curves.** For each group, estimate the share built and the share withdrawn by years since queue entry (Aalen-Johansen method).
   - Why: projects end in two ways, and many are still waiting. A simple built/total ratio ignores the waiting ones. Treating withdrawal as a censored outcome overstates build-out.
3. **Backtest.** Pretend it's the end of 2019: hide everything after that date, fit the model, predict 2020 to 2025 outcomes, then compare with what actually happened.
   - Why: the only honest test of a forecaster is checking it against outcomes it couldn't see.
4. **Monte Carlo outlook.** For each waiting project, draw whether and when it gets built, given how long it has already waited. Sum MW by region and year. Repeat 10,000 times.
   - Why: it gives ranges, not one number. Conditioning on time already waited matters, because a project 6 years in has different odds than one queued last month.

The backtest gates the forecast



model pipeline · 4 stages, 1 gate

Only a calibrated model feeds the capacity outlook; a failed backtest loops back to Stage 2.

Milestones

Each day ends with an exit check. A day isn't done until its check passes and Kevin has explained it back.

Day	Date	Deliverable	Exit check
1	Wed Oct 21	Clean dataset and data dictionary	Counts match the report; decisions D1 to D4 agreed
2	Thu Oct 22	Build and withdrawal curves by technology and region	Plots plus 3 plain-English findings
3	Fri Oct 23	Backtest and Monte Carlo outlook	Calibration plot with verdict; P10/P50/P90 table
4	Sat Oct 24	README, tests, public repo	Tests pass; repo public; résumé bullet filled with real numbers

Cut line: if Day 2 isn't done by end of Thursday, drop the state-level outlook and size bands. Keep technology and region only.

Decision log

Every modelling choice lives here with its reason. This table is your interview prep.

#	Decision	Recommendation	Why	Status
D1	Start of the clock	Queue entry date	Same starting point for every project	Proposed
D2	What counts as built	Status operational, with an operational date	It's the only outcome that delivers power	Proposed
D3	Suspended projects	Treat as still waiting; rerun treating them as withdrawn	Suspended projects can resume; showing both bounds the effect	Proposed

#	Decision	Recommendation	Why	Status
D4	End of observation	The data snapshot date, end of 2025	It's the last date we actually know outcomes	Proposed
D5	Core model	Aalen-Johansen curves by group; cause-specific Cox model for size and entry year	Handles two competing outcomes and stays explainable	Proposed
D6	Backtest cutoff	Fit as of end of 2019, score on 2020 to 2025	Six years of real outcomes to check against	Proposed
D7	Groups	Technology by region; merge groups under 100 projects	Matches how Berkeley Lab reports; avoids noisy tiny groups	Proposed
D8	Simulation runs	10,000	Enough for stable P10 and P90; verify by rerunning with a new seed	Proposed
D9	Who writes code	Claude writes; Kevin decides, reviews and explains back	Fits the 4-day deadline and the TPM story	Agreed

Risks

The biggest risk is missing outcome dates in the data file, so that gets checked in the first hour of Day 1.

Risk	Impact	Mitigation
Outcome dates missing or sparse	No survival analysis possible	Check first; fallback is joining yearly editions, which costs a day, so the state-level work gets cut
Status rules differ by region	Unfair regional comparisons	One documented mapping rule; report at region level only

Risk	Impact	Mitigation
Recent projects haven't finished yet	Naive rates understate build-out	Survival methods handle this; flag it in the README
Queue reforms change behaviour (for example, FERC Order 2023 cluster studies)	Past odds may not hold	The backtest measures drift; state it as a limitation
Small groups	Noisy curves	Merge groups under 100 projects (D7)
Overclaiming novelty	Credibility hit	README says plainly what Berkeley Lab already reports
4-day deadline slips	No finished résumé claim	Cut line in Milestones

Setup and working agreement

We build in Claude Code inside VS Code, so the repo, files and git history live on Kevin's laptop.

Dependencies

- [ ] Kevin downloads the Berkeley Lab project-level data file
- [ ] Install VS Code, Python 3.11+, git and the Claude Code extension
- [ ] Create an empty GitHub repo named grid-queue-forecaster

Stack: Python, pandas, lifelines, matplotlib, pytest.

Repo layout

```
grid-queue-forecaster/
  data/raw/          # Berkeley Lab file, not committed
  data/clean/        # cleaned dataset
  src/clean.py       # Stage 1
  src/survival.py    # Stage 2
  src/backtest.py    # Stage 3
  src/simulate.py    # Stage 4
  tests/
  docs/PLAN.md       # this plan
  README.md
```

Daily loop

1. Claude lays out that day's decisions; Kevin picks.
2. Claude writes and runs the code.
3. Walkthrough focused on why, block by block.
4. Kevin explains each stage back in 2 to 3 sentences and owns one small piece (a rule or a test).
5. Update the decision log and commit.

Honesty rule: the résumé only claims finished stages with real numbers. If asked, Kevin says Claude wrote the code and he scoped it, made the modelling calls and validated the results.

Sources

- Berkeley Lab, Queued Up 2026 Edition (data page)
- Queued Up 2026 Edition report (PDF)